Predicting cp.async Pipeline Effectiveness from Memory-Subsystem Parameters: A Cross-GPU Study

Xiangchen Song
University of Michigan – Ann Arbor
xxxchen@umich.edu

Chenxin Geng
University of Michigan – Ann Arbor
gchenxin@umich.edu

Yuyang Feng
University of Michigan – Ann Arbor
yuyangfe@umich.edu

Tian Xia
University of Michigan – Ann Arbor
crazyjoe@umich.edu

Abstract

NVIDIA's cp.async instruction enables asynchronous global-to-shared memory copies that can be software-pipelined with computation, and is now standard in high-performance GPU kernels. Practitioners typically tune pipeline depth per kernel and assume the result transfers across GPUs sharing the same instruction. We test this assumption by running *identical* cp.async pipelined kernels—FP32 GEMM and a 1D stencil—on three GPUs that support cp.async but have radically different memory subsystems: A40 (GDDR6, 6 MB L2, 696 GB/s, ridge = 53.7), A100 MIG 3g.40gb (HBM2e, 20 MiB L2, 968 GB/s, ridge = 7.9), and H100 SXM5 (HBM3, 50 MB L2, 3350 GB/s, ridge = 20.0). We build a parameter-free predictive model from architecture constants and microbenchmarked latencies, validate it across all three GPUs and both kernels, and package it as a lightweight Triton autotune prefilter that prunes the pipeline-depth search space. A pointer-chasing microbenchmark characterizes each GPU's effective L2 behavior—including an empirical characterization of MIG-instance L2 capacity and latency, which is not specified in vendor documentation—and feeds directly into the predictor.

1 Problem Definition and Research Question

Hiding global memory latency is central to GPU kernel optimization. NVIDIA's cp.async (Ampere and later) enables asynchronous global-to-shared copies that bypass registers and can be software-pipelined with computation [8]. Libraries such as CUTLASS 3.x [11] and FlashAttention-3 [9] rely heavily on multi-stage cp.async pipelines, and autotuners (including Triton's [12]) sweep pipeline depth S as a key tuning knob.

However, pipeline effectiveness is not purely an ISA property. It depends on the interaction between L2 cache capacity (determining whether tile data is served from cache or DRAM), memory bandwidth and latency (determining how long a copy stalls), and per-SM shared memory (determining how many pipeline stages fit before occupancy degrades). These parameters vary dramatically even among GPUs that share cp.async support—yet no study has quantified how this variation affects optimal pipeline configuration, and no existing autotuner uses architecture parameters to prune the search space.

Research Question: *For an identical* cp.async *pipeline kernel, does the optimal pipeline depth S^* and its marginal speedup diverge across GPUs with different memory subsystems? Can a lightweight, parameter-free model predict this divergence from architecture constants alone—and can such a model reduce autotuning cost in practice?*

2 Hypotheses

We test three competing explanations for how memory-subsystem parameters govern cp.async pipeline effectiveness. Each hypothesis makes distinct, falsifiable predictions about the observables defined in Section 7.

- **H1 (L2-capacity-driven):** Pipeline speedup exhibits a sharp inflection when the concurrent working set W_{conc} exceeds L2 capacity C_{L2}. Plotting speedup against $W_{\text{conc}}/C_{\text{L2}}$ collapses all three GPU curves onto a single master curve. *Falsifiable prediction:* after L2 normalization, A100-MIG (20 MiB) and H100 (50 MB) should *not* overlap—if they do despite a 2.4× L2 gap, L2 capacity alone is insufficient.
- **H2 (Overlap-saturation / ridge-point-driven):** Pipeline speedup is governed by whether per-tile compute time can mask copy latency, i.e. by the ridge point $R = F_{\text{peak}}/\text{BW}_{\text{peak}}$. GPUs with low R (more memory-bound) benefit most from pipelining. *Falsifiable prediction:* A100-MIG ($R = 7.9$, lowest of the three) should show the largest pipeline speedup; A40 ($R = 53.7$) the smallest—possibly even a slowdown if occupancy loss outweighs negligible latency-hiding benefit.
- **H3 (Latency–occupancy tension):** Optimal depth S^* results from the tension between memory latency (favoring deeper pipelines) and shared-memory capacity (limiting depth through occupancy loss). A40 has the highest expected DRAM latency (GDDR6; latency to be confirmed by microbenchmark, Section 5) yet the smallest per-SM shared memory (100 KB). *Falsifiable prediction:* $S^*(N)$ is

non-monotonic on A40 (rises then falls with N), while remaining monotone or flat on A100-MIG and H100 (more SMEM headroom).

Discrimination logic. H1: curve-collapse under L2 normalization. H2: speedup rank follows ridge-point rank, not L2 rank. H3: $S^*(N)$ exhibits GPU-dependent non-monotonicity. In practice a mixture is likely; the predictive model (Section 8) incorporates all three mechanisms and the *pipeline benefit map* (Section 7.1) visualizes their interplay across the full (AI, W/C_{L2}) space.

3 Related Work

Roofline model. Williams et al. [15] introduced the roofline model relating arithmetic intensity to attainable performance. We use it as the primary cross-GPU normalization framework, computing per-GPU roofline ceilings from runtime-queried hardware parameters (including MIG-instance peaks).

`cp.async` pipeline studies. Li et al. [5] demonstrate >2× speedup from double-buffered `cp.async` pipelines for SpMM on Ampere (ASPLOS 2024); our V3 adopts a similar multi-stage pattern but sweeps stage count across three GPUs and two kernels rather than one. Chen et al. [1] (Tawa, 2025) sweep pipeline depth on H100, revealing occupancy–depth tradeoffs that motivate H3; we test whether these tradeoffs shift on GPUs with smaller SMEM. Wu et al. [16] (Twill, 2025) note that optimal pipelining relies on "brittle heuristics"; our Triton pre-filter (Section 9) targets a complementary, spec-based alternative.

GPU L2 cache and tile scheduling. Wei et al. [14] (HyTiS, SC 2025) model L2-aware tile scheduling for GEMM, showing up to 36% throughput variation from tile placement on A100/H100, but do not study async pipeline interaction at L2 boundaries. Jin et al. [4] (MICRO 2024) reverse-engineer GPU NoC topology and find up to 70% L2 latency variation from SM-to-slice distance. Ernst et al. [2] identify GPU cache "layer conditions" where performance shifts qualitatively at capacity thresholds. No prior work characterizes L2 cache behavior under MIG partitioning.

GPU microbenchmarking. Mei and Chu [6] establish pointer-chasing methodology for GPU cache hierarchy characterization. Wang et al. [13] microbenchmark Hopper's TMA and L2 on H100 only. We extend pointer-chasing to MIG instances, measuring effective (not nominal) L2 capacity and latency under partitioned resources.

Cross-GPU benchmarking. Svedin et al. [10] benchmark K80 through A100 on Rodinia (2021) but predate H100, lack roofline framing, and omit A40. No prior work compares the *same* `cp.async` pipeline kernel across GPUs with different memory subsystems.

Analytical models and autotuning. Guo et al. [3] (tritonBLAS, 2025) propose an analytical kernel-parameter selector for Triton GEMM but target a single GPU. ACTA [17] models TMA configuration on Hopper. Triton's `@triton.autotune` [12] performs exhaustive grid search over all configurations including `num_stages`. Our model is simpler (overlap + occupancy penalty) but designed to *transfer across GPUs* and to *prune the autotune search space* before exhaustive search begins.

Table 1. Target GPUs. A100 values are runtime `cudaDeviceProp` measurements from the MIG 3g.40gb instance. All other GPUs are full devices.

	V100	**A40**	**A100 MIG 3g**	**H100**
Cluster	Great Lakes	Great Lakes	Great Lakes	Lighthouse
Arch / SM ver.	Volta / 70	Ampere / 86	Ampere / 80	Hopper / 90
SMs	80	84	42	132
`cp.async`	—	✓	✓	✓
Memory tech	HBM2	GDDR6	HBM2e (MIG)	HBM3†
GPU memory	32 GB	48 GB	40 GB	80 GB
Bus width	4096 bit	384 bit	2560 bit	5120 bit
Peak BW	900 GB/s	696 GB/s	968 GB/s	3350 GB/s
L2 cache	6 MB	6 MB	20 MiB‡	50 MB
SMEM/SM	96 KB	100 KB	164 KB	228 KB
FP32 peak	15.7 TF	37.4 TF	7.6 TF	67 TF
Ridge (FLOP/B)	17.4	53.7	7.9	20.0
Role	Boundary	Core anchor	Intermediate	Core anchor

†H100 on Lighthouse is the SXM5 variant (HBM3, 3.35 TB/s).
‡Reported by `cudaDeviceProp`; effective capacity $C_{\mathrm{L2}}^{\mathrm{eff}}$ measured via microbenchmark.

4 Experimental Setup

4.1 Hardware and Experimental Roles

The four GPUs (Table 1) play distinct roles:

- **A40 and H100 (core contrast pair, both full GPUs):** Maximum memory-subsystem divergence among `cp.async`-capable GPUs: 8.3× L2, 4.8× BW, 2.7× ridge point—with no MIG artifacts.
- **A100 MIG 3g.40gb (intermediate + ridge-point extreme):** MIG partitions compute more aggressively than bandwidth (42/108 = 39% SMs, 50% memory bus), producing the *lowest* ridge point (7.9) of any GPU in the study. H2 predicts it should benefit *most* from pipelining. Also serves as a real-world test of predictor robustness under MIG-partitioned resources [7].
- **V100 (boundary condition):** Lacks `cp.async`; runs V0–V1 only. The predictor must output speedup = 1.0×.

GPU pair separation logic. The three `cp.async`-capable GPUs span a wide, non-aligned region of (L2, BW, ridge) space:

Pair	**ΔL2**	**ΔBW**	**ΔRidge**	**Primary contrast**
A40 vs. H100	8.3×	4.8×	2.7×	Max. memory-subsystem gap
A40 vs. A100-MIG	3.3×	1.4×	6.8×	Same ISA; ridge-point extreme
A100-MIG vs. H100	2.4×	3.5×	2.5×	Intermediate → high

Table 2. Kernel variants. V3-GEMM and V3-Stencil are the `cp.async` pipeline kernels central to the study.

ID	GPUs	Definition
V0-GEMM	All	Naive global load, no tiling.
V1-GEMM	All	Shared-memory tiling, synchronous load + `__syncthreads`.
V3-GEMM	A40+	Multi-stage `cp.async` pipeline, $S \in \{2, 3, 4\}$. Tile geometry matches V1.
V1-Stencil	All	1D stencil, synchronous shared-memory tiling with halo.
V3-Stencil	A40+	Same stencil with `cp.async` pipelined loads, $S \in \{2, 3, 4\}$.
Ref	All	cuBLAS SGEMM (ceiling / sanity).

No two parameters are monotonically ordered across all three GPUs (A100-MIG has the *lowest* ridge despite mid-range L2), so the predictor cannot succeed by fitting a single monotonic trend.

4.2 Kernel Variants

We implement two kernel families (Table 2):

FP32 GEMM ($C = A \times B$, row-major) with matched tile structures across V0, V1, and V3. GEMM has high arithmetic intensity (AI $\gg R$ for all GPUs at $N \geq 512$), making it compute-bound in the roofline sense but still subject to per-tile memory stalls that pipelining can hide.

1D Stencil with a 5-point stencil on a 1D array of length L. Stencil has much lower AI (~5 FLOP/B after tiling), placing it near or below the ridge point of most GPUs. This makes it genuinely memory-bound and provides a complementary test: if the predictor works for both a compute-bound kernel (GEMM) and a memory-bound kernel (stencil), the model captures general memory-subsystem properties rather than GEMM-specific behavior. To ensure the pipeline has sufficient compute to overlap with async copies (rather than being dominated by barrier overhead), the stencil kernel uses loop unrolling and multiple elements per thread to increase per-stage compute density.

The **same source code** for V3-GEMM (and V3-Stencil) compiles and runs on A40, A100-MIG, and H100 with no GPU-specific branches.

4.3 Workload and Measurement

GEMM: $N \in \{256, 512, 1024, 2048, 3072, 4096, 6144, 8192\}$. For the *pipeline benefit map* (Section 7.1), we additionally sweep tile sizes $T \in \{16, 32, 64, 128\}$ to vary per-tile AI.

Stencil: $L \in \{2^{16}, 2^{18}, 2^{20}, 2^{22}, 2^{24}, 2^{26}\}$.

Timing: `cudaEvent_t`, 3 warmup + 10 timed iterations, median. Correctness: verified against reference (cuBLAS for GEMM; CPU for stencil).

5 MIG L2 Cache Microbenchmark

NVIDIA documents that MIG partitions memory by slice, but does not publish effective L2 capacity, associativity, or replacement behavior under MIG [7]. The `cudaDeviceProp`-reported 20 MiB may not reflect the *usable* capacity if MIG boundaries cause additional conflict misses or if L2 partition granularity differs from the nominal slice assignment.

We run a **pointer-chasing microbenchmark** (following Mei and Chu [6]) on every GPU to measure:

1. **L2 effective capacity ($C_{\mathbf{L2}}^{\mathbf{eff}}$):** Sweep array size from 1 MB to 40 MB; identify the inflection where average load latency jumps from L2-resident to DRAM-resident. On A100-MIG, if $C_{\mathrm{L2}}^{\mathrm{eff}} < 20$ MiB, MIG imposes hidden overhead.
2. **L2 hit latency ($L_{\mathbf{L2}}$) and DRAM latency ($L_{\mathbf{DRAM}}$):** Measured in cycles at plateau regions below and above $C_{\mathrm{L2}}^{\mathrm{eff}}$. These are direct inputs to the predictive model (Section 8).
3. **L2 bandwidth curve:** Streaming bandwidth vs. working-set size, per GPU. Reveals whether MIG's L2 partition sustains the same bandwidth per byte as the full die.

These measurements are treated as *per-GPU constants* (not fitted from GEMM data) and are performed once during Week 1–2. If $C_{\mathrm{L2}}^{\mathrm{eff}}$ differs from the reported 20 MiB on the MIG instance, this provides an empirical characterization of MIG-instance effective L2 behavior—information not specified in vendor documentation [7].

6 Working-Set Definition and L2 Analysis

For a tiled GEMM with tile dimensions $T_M \times T_K$ and $T_K \times T_N$, the per-block working set is:

$$W_{\mathrm{block}} = (T_M \cdot T_K + T_K \cdot T_N) \times 4 \text{ bytes (FP32)}.$$

The concurrent working set across simultaneously active blocks sharing the L2:

$$W_{\mathrm{conc}} = W_{\mathrm{block}} \times S \times \min\big(B_{\mathrm{active}},\ \lceil N/T_M \rceil \cdot \lceil N/T_N \rceil\big),$$

where S is the pipeline stage count and $B_{\mathrm{active}} = N_{\mathrm{SM}} \times \lfloor S_{\mathrm{SM}}/(W_{\mathrm{block}} \cdot S) \rfloor$ is the occupancy-limited concurrency.

For the stencil, W_{block} includes the tile plus halo elements; the same W_{conc} framework applies with adjusted tile geometry.

In both cases, we use the *microbenchmarked* $C_{\mathrm{L2}}^{\mathrm{eff}}$ (Section 5) rather than the nominal `cudaDeviceProp` value when computing $W_{\mathrm{conc}}/C_{\mathrm{L2}}$.

7 Key Plots and Hypothesis Discrimination

1. **Speedup vs. N (raw), per kernel:** Pipeline speedup for A40, A100-MIG, H100 on the same plot. Separate plots for GEMM and stencil. If all three curves separate, "pipeline tuning transfers" is falsified.
2. **Speedup vs. $W_{\mathbf{conc}}/C_{\mathbf{L2}}^{\mathbf{eff}}$ (L2-normalized):** Tests H1 (collapse $\Rightarrow$ L2 explains all). Uses microbenchmarked effective L2, not nominal.
3. **$S^*(N)$ per GPU:** Optimal pipeline depth as a function of problem size. Tests H3 (non-monotonicity on A40).

4. **Speedup rank vs. ridge-point rank:** Tests H2 (A100-MIG > H100 > A40 ordering).
5. **Model prediction vs. measurement (scatter):** x = predicted, y = measured, one point per (N, g, S) triple. MAPE per GPU and per kernel.

7.1 Pipeline Benefit Map

Beyond the above plots, we produce a **two-dimensional heatmap** for each GPU:

- x-axis: per-tile arithmetic intensity (varied by sweeping tile size $T \in \{16, 32, 64, 128\}$ in GEMM);
- y-axis: $W_{\text{conc}}/C_{\text{L2}}^{\text{eff}}$ (varied by sweeping N);
- color: pipeline speedup $T_{V1}/T_{V3_{\text{best}}}$.

This map answers the practical question: *for a kernel with a given AI and data footprint, on this GPU, is pipeline worth enabling?* The shape of the "pipeline-beneficial" region (green) differs across GPUs: on A40 (ridge = 53.7) only the low-AI, high-W/C corner should be green; on A100-MIG (ridge = 7.9) most of the map should be green. Comparing the three maps provides a visual summary of all three hypotheses simultaneously.

7.2 Profiling Signals

Nsight Compute metrics collected at each (N, g, S): L2 hit rate, DRAM bytes read, achieved occupancy, barrier stall cycles, and L2 throughput utilization. These provide mechanistic support: a speedup inflection aligned with L2-hit-rate drop supports H1; speedup tracking compute-to-copy ratio (barrier stalls) supports H2; non-monotonic S^* reversing when occupancy drops supports H3.

7.3 Anomaly Analysis Protocol

After collecting all data, we compute $\Delta(N, g, S) = \widehat{\text{Speedup}} - \text{Speedup}_{\text{measured}}$ for every configuration and rank by $|\Delta|$. The top-5 largest deviations receive a targeted Nsight Compute deep-dive: full metric collection at that specific (N, g, S) to identify the root cause of model error.

Possible anomaly sources include: MIG L2 partitioning artifacts (addressed by Section 5); L2 bank conflicts at specific tile geometries (cf. [4]); occupancy cliff effects where one additional SMEM buffer crosses a block-per-SM boundary. **Documenting model failure modes is itself a contribution**—it identifies which architectural effects a simple overlap model misses and where more detailed modeling is needed.

8 Predictive Model (Parameter-Free)

We construct a semi-analytic predictor of `cp.async` pipeline speedup combining an L2-aware latency estimate with an occupancy penalty, following the overlap reasoning of Little's Law (cf. [17]).

8.1 Effective Memory Latency

Let h be the estimated L2 hit fraction:

$$h(N, g) = \min\big(1,\ C_{\text{L2}}^{\text{eff}}(g)\ /\ W_{\text{conc}}(N, g)\big).$$

Note: $C_{\text{L2}}^{\text{eff}}$ is the *microbenchmarked* effective capacity (Section 5), not the nominal device-property value. Our h is a capacity-based upper bound on the true hit rate; real L2 behavior is further affected by set-associativity, slice locality, and replacement policy [4]. Deviations between this approximation and reality are explicitly analyzed via the anomaly protocol (Section 7.3) and Nsight Compute L2 hit-rate metrics. The effective latency seen by a tile load is:

$$L_{\text{eff}}(N, g) = h \cdot L_{\text{L2}}(g) + (1 - h) \cdot L_{\text{DRAM}}(g),$$

where L_{L2} and L_{DRAM} are microbenchmarked constants.

8.2 Predicted Speedup

Minimum depth to fully hide latency:

$$S_{\min}(N, g) = \left\lceil \frac{L_{\text{eff}}(N, g)}{T_{\text{compute}}(g)} \right\rceil,$$

where T_{compute} is per-tile pure compute time, isolated from memory stalls via one of two methods: (i) Nsight Compute's `smsp__cycles_active` metric on V1 minus memory-stall cycles, or (ii) a compute-only microkernel that operates on data already resident in shared memory / registers. Using raw V1 wall-clock time would conflate memory stalls with compute, biasing $S_{\min}$; isolating pure compute avoids this circularity. Predicted speedup:

$$\widehat{\text{Speedup}}(N, g, S) = \frac{\max\big(T_{\text{compute}},\ L_{\text{eff}}\big)}{\max\big(T_{\text{compute}},\ L_{\text{eff}}/\min(S,\ S_{\min})\big)} \times \frac{\text{Occ}(S, g)}{\text{Occ}(1, g)}.$$

The occupancy ratio $\text{Occ}(S)/\text{Occ}(1)$ is computed from the CUDA occupancy API given measured SMEM and register usage—not fitted.

8.3 Why "Parameter-Free" Despite Few GPUs

Every model input is either an architecture constant ($C_{\text{L2}}^{\text{eff}}$, BW, F_{peak}, S_{SM}, N_{SM}) or a per-GPU microbenchmark constant (L_{L2}, L_{DRAM}). No free parameters are fitted from GEMM or stencil results. The kernel experiments serve to *validate*, not fit. Demonstrating accuracy on two distinct kernels (GEMM and stencil) across three GPUs further guards against overfitting to a single workload.

8.4 Validation Protocol

1. **Per-GPU accuracy:** predicted vs. measured speedup for all (N, S) on each GPU. Report MAPE per GPU, per kernel.
2. **Boundary condition:** V100 $\longrightarrow$ 1.0× by construction.
3. **Leave-one-out:** hide one GPU's *measurements*, predict from its architectural parameters alone, report held-out MAPE.
4. **Nominal vs. effective L2:** compare MAPE when using $C_{\text{L2}}^{\text{eff}}$ (microbenchmarked) vs. $C_{\text{L2}}^{\text{nom}}$ (`cudaDeviceProp`). If

microbenchmarked values improve accuracy, the L2 characterization has direct predictive value.

9 Triton Autotune Pre-Filter

We package the predictive model as a **lightweight pre-filter for Triton's `@triton.autotune` decorator** [12].

Current Triton behavior. Triton's autotune exhaustively benchmarks every configuration in the user-specified grid, including all `num_stages` values. For a typical GEMM tuning grid with 4 tile sizes × 3 stage counts × 2 `num_warps` settings = 24 configurations, each evaluated with warmup + timed iterations, autotuning can take 30–60 seconds per kernel launch shape.

Our pre-filter. Before autotune begins, query the GPU's architectural parameters (via `cudaDeviceProp` and our microbenchmarked latencies), compute the predicted speedup for each `num_stages` candidate, and *prune* candidates whose predicted speedup is within a threshold ϵ of 1.0× (i.e., pipeline unlikely to help).

Expected outcome. On A40 (ridge = 53.7), the pre-filter should prune $S \in \{3, 4\}$ for large N (occupancy cost exceeds benefit), reducing the search space by ∼50%. On A100-MIG (ridge = 7.9), few or no candidates are pruned (pipeline is beneficial). We measure: (i) wall-clock autotune time with and without pre-filter, (ii) whether pruning ever removes the true optimal configuration (false-negative rate), and (iii) end-to-end kernel performance (must not degrade).

Implementation. ∼150 lines of Python wrapping the model equations from Section 8, invoked as a decorator that wraps `@triton.autotune`. Evaluated on Triton's tutorial GEMM kernel and (if time permits) on a Triton stencil kernel.

10 Milestones, Schedule, and Division of Labor

Member roles: A build system, baselines, MIG validation, pointer-chasing microbench; **B** V3-GEMM and V3-Stencil implementation, A40/A100-MIG runs, tile-size sweep; **C** H100 coordination (Lighthouse), stencil runs, Triton pre-filter; **D** Nsight profiling, L2/working-set analysis, model implementation, heatmap generation, anomaly analysis. All members contribute to writing and poster.

Decision point (Week 6). If GEMM speedups converge across GPUs (no divergence), the "pipeline tuning is transferable" conclusion is itself useful; we then invest more effort in the stencil kernel (lower AI, higher chance of divergence) and the MIG L2 characterization. If divergence is strong, we prioritize the predictor, heatmap, and Triton integration.

Contributions at a glance.

1. A controlled cross-GPU `cp.async` pipeline comparison (same code, three memory subsystems, two kernels).
2. An empirical characterization of MIG-instance effective L2 behavior (capacity and latency under pointer-chasing), not specified in vendor documentation.
3. A parameter-free predictor validated on two kernels and three GPUs (including MIG).
4. A pipeline benefit map visualizing the (AI, W/C_{L2}) region where pipelining helps, per GPU.
5. A Triton autotune pre-filter prototype demonstrating practical search-space reduction.

Table 3. Timeline and responsibilities. Items marked ⋆ are stretch goals that enhance novelty but are not required for the core contribution.

Week	Milestone	Lead
1–2	Build system (CMake, SLURM). V0/V1 baselines on all 4 GPUs. MIG device-query validation. **Pointer-chasing microbench** (L_{L2}, L_{DRAM}, $C_{\mathrm{L2}}^{\mathrm{eff}}$) on all GPUs.	A + C
3–4	V3-GEMM ($S = 2, 3, 4$) + V1/V3-Stencil implementation. Initial speedup curves on A40, A100-MIG.	B
5–6	Full A40 / A100-MIG / H100 GEMM comparison. Nsight profiling. Tile-size sweep for heatmap data. *Milestone 1 slides.*	B + D
7–8	Stencil experiments on all GPUs. Predictor implementation + validation. Anomaly deep-dives. ⋆Triton pre-filter prototype.	C + D
9–10	Pipeline benefit maps. Final figures, hypothesis conclusions. Report + poster.	All

References

[1] Hongzheng Chen, Rohan Yadav, Jianfei Gao, Hasan Ferdous, Jianan Gu, Amir Hossein Nodehi Sabet, Ce Jin, Michael Garland, and Kunle Olukotun. 2025. Tawa: Automatic Warp Specialization for Modern GPUs with Asynchronous References. *arXiv preprint arXiv:2510.14719* (2025).

[2] Dominik Ernst, Georg Hager, Jonas Thürner, and Gerhard Wellein. 2023. Analytical Performance Estimation during Code Generation on Modern GPUs. *J. Parallel and Distrib. Comput.* 173 (2023), 152–163.

[3] Jiarui Guo et al. 2025. tritonBLAS: Triton-based Analytical Approach for GEMM Kernel Parameter Selection. *arXiv preprint arXiv:2512.04226* (2025).

[4] Jingwen Jin, Yifan Sun, and Tor M. Aamodt. 2024. Uncovering Real GPU NoC Characteristics: Reverse-Engineering and Simulation Validation. In *Proceedings of MICRO*.

[5] Guohao Li, Zhekai Zheng, Yifei Rong, and Luis Ceze. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of ASPLOS*.

[6] Xinxin Mei and Xiaowen Chu. 2017. Dissecting GPU Memory Hierarchy through Microbenchmarking. In *IEEE Transactions on Parallel and Distributed Systems*, Vol. 28. 72–86.

[7] NVIDIA. 2024. NVIDIA Multi-Instance GPU User Guide. https://docs.nvidia.com/datacenter/tesla/mig-user-guide/.

[8] NVIDIA. 2025. CUDA C++ Programming Guide, v12.8: Asynchronous Data Copies. https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[9] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. In *Proceedings of NeurIPS*.

[10] Martin Svedin, Steven Westergren Norgren, Hao Lu, Artur Podobas, and Stefano Markidis. 2021. Benchmarking the NVIDIA GPU Lineage: from K80 to A100 with Application to HPC Solvers. In *Proceedings of HEART*.

[11] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Haicheng Lu, Ethan Yan, Jack Kosaian, and Mark Nicely. 2023. CUTLASS. https://github.com/NVIDIA/cutlass. NVIDIA.

[12] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. 10–19.

[13] Qiang Wang, Weile Luo, and Xiaowen Chu. 2024. Dissecting the NVIDIA Hopper Architecture through Microbenchmarking and Multiple Level Analysis. In *Proceedings of IPDPS*.

[14] Yihua Wei, Zihan Liu, Xieyu Zhong, Licheng Guo, and Jason Cong. 2025. HyTiS: Hybrid Tile Scheduling for GPU GEMM with Enhanced Wave Utilization and Cache Locality. In *Proceedings of SC*.

[15] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (2009), 65–76.

[16] Haicheng Wu, Jian Weng, et al. 2025. Optimal Software Pipelining and Warp Specialization for Tensor Core GPUs. *arXiv preprint arXiv:2512.18134* (2025).

[17] Jingqi Xu, Guowei Zhang, et al. 2025. ACTA: Automatic Configuration of the Tensor Memory Accelerator for High-End GPUs. In *Proceedings of the 17th Workshop on General Purpose Processing Using GPU (GPGPU)*.